GPU 向量检索引擎（CUDA）总结报告

状态：已在 NVIDIA GeForce RTX 4090 24GB 上完成 Release 构建、正确性测试和百万规模性能矩阵。表中数字均来自 results/*.json，未使用估算或示例值。

1. 问题描述

本项目面向 RAG、推荐系统和多模态检索中的 Top-K 向量搜索。给定 N 条 D 维数据库向量和一批查询向量，系统返回 L2 距离最小或内积最大的 K 条记录。精确搜索复杂度为 O(ND)，可以作为正确性基准；IVF-Flat 通过 coarse quantization 只访问部分倒排桶，以召回率换取吞吐和延迟。

2. 完成内容

项目实现 CPU 精确 baseline、CUDA 精确距离计算、批量查询、排序 Top-K、L2/内积、IVF-Flat
索引、索引保存与加载、二进制输入输出、正确性测试和统一指标日志。支持任意正整数 K，正式脚本覆盖 K=1、10、50、100。

基础实现使用 FP32。IVF 聚类、coarse search 和最终 Top-K 在 CPU 完成，数据库常驻 GPU，候选 ID 上传后由 CUDA kernel 读取原始向量并计算距离。本项目未实现 IVF-PQ、GPU Top-K、FP16 或国产 GPU 适配，不将这些未完成项写成已完成。

3. 数据布局与文件格式

数据库和查询采用 row-major 连续 FP32 数组。相邻 CUDA
线程处理相邻向量；固定维度循环中，相邻线程的全局内存访问存在跨向量步长。此布局与题目文件定义一致，容易验证和落盘，但不是所有维度下最理想的 coalescing 布局。后续可尝试转置或 tiled layout。

文件带 8 字节 magic 和版本号，并固定整数字段宽度。IVF 索引文件保存聚类中心、原始向量和倒排列表；加载时校验形状、总 ID 数和 ID 范围，避免每次查询重新训练。

4. 精确检索算法

CPU 参考实现逐 query 计算所有距离，并用 std::partial_sort 得到排序 Top-K。CUDA kernel 使用二维 grid，一维覆盖 N 条候选，一维覆盖 batch。一个线程累加一个 query-vector pair 的 D 个维度，L2 和 inner product 均使用 fmaf。内积取负后复用升序排序逻辑，输出时恢复真实相似度。

GPU 结果将距离复制回主机，在 CPU 完成 Top-K。这是当前版本最明确的性能边界：距离阶段在 GPU，但 distance D2H 与 CPU partial sort 仍可能成为瓶颈。

5. IVF-Flat

5.1 建索引

1. 从数据库随机选取初始中心。
2. 使用有固定上限的 mini-batch Lloyd 迭代更新中心，避免百万规模训练遍历成本失控。
3. 将全部向量分配给最近中心，建立倒排桶。
4. 保存中心、数据库向量和每个桶的原始 ID。

5.2 查询

1. 在 CPU 计算 query 与所有 coarse centroid 的距离。
2. 选取最近的 nprobe 个桶。
3. 仅把候选 ID 传入 GPU，数据库在一次 IVF 查询过程中常驻显存。
4. CUDA kernel 按 ID 读取原始向量并计算精确距离。
5. CPU Top-K 归并并映射回原始 ID。

nprobe 越大，通常 recall@K 越高，但候选数、延迟和工作区也更大。正式脚本测试 1、4、16、64 四档。

6. 正确性验证

自动测试覆盖非 2 的幂维度 D=33，以避免只在整齐 shape 下通过；分别验证 L2 和 inner product。精确 GPU 与 CPU 的 Top-K ID 集合要求完全一致，分数误差容限为 2e-3。另有 clustered synthetic data 的 IVF 索引 round-trip 测试，要求 recall@10 不低于 0.90。

检查项	当前结果
CMake Release 构建	通过
ctest	1/1 通过（0.34 s）
CPU vs GPU exact recall@K	1.000000
L2 Top-K/分数测试	通过（容限 2e-3）
Inner product Top-K/分数测试	通过（容限 2e-3）
IVF save/load	通过

7. 实验设置

项目	值
GPU	NVIDIA GeForce RTX 4090
GPU 显存	24,564 MiB
CPU	Intel Xeon Platinum 8352V @ 2.10 GHz，16 vCPU
OS	Ubuntu 22.04 / Linux 5.15.0-91-generic
NVIDIA Driver	580.95.05
CUDA Toolkit / nvcc	11.8 / V11.8.89
编译器 / CMake	G++ 11.3.0 / CMake 3.22.1
编译类型	Release (-O3, CUDA --use_fast_math)
数据规模	1,000,000 x 128，1000 queries
随机种子	42

正式计时关闭了其他 GPU 任务。精确 QPS 包含 query H2D、CUDA 距离、distance D2H 和 CPU Top-K，不包含数据库首次上传；IVF QPS 还包含 CPU coarse search、候选 ID 构造和一次数据库 H2D。两者都不包含索引文件加载。P50/P99 用 batch=1 的逐 query 精确查询时间。显存字段是程序显式 buffer 大小，不包含 CUDA context。

8. 性能与质量结果

8.1 精确检索与 K

K	Batch	QPS	P50 (ms)	P99 (ms)	CPU/GPU 加速比	CPU/GPU recall@K	工作区 MiB
1	128	263.571	4.622	6.801	42.444x	1.000000	976.625
10	128	261.783	4.650	6.038	42.262x	1.000000	976.625
50	128	274.503	4.281	5.897	42.709x	1.000000	976.625
100	128	255.570	4.402	5.988	40.381x	1.000000	976.625

8.2 Batch size

Batch	QPS	P50 (ms)	P99 (ms)	工作区 MiB
1	231.165	4.391	6.144	492.096
32	276.272	4.874	6.491	610.367
128	261.783	4.650	6.038	976.625

8.3 IVF nprobe

nlist	nprobe	QPS	recall@10	平均排名分数误差	工作区 MiB
256	1	3509.281	0.059600	14.431664	488.390
256	4	1986.191	0.182900	8.026410	488.680
256	16	595.972	0.472900	3.293365	489.774
256	64	153.481	0.937800	0.272549	493.920

9. 结果分析

batch=32 在本次运行中取得最高精确检索吞吐 276.272 QPS，比 batch=1 的 231.165 QPS 高约 19.5%；batch=128 未继续提升，说明更大的 distance buffer 和 CPU Top-K 并未带来额外收益。P50/P99 都在约 4.3-6.5 ms 范围内，差异接近单次运行波动。

K 从 1 增加到 100 后，精确 QPS 仍处于 255-275 区间，未呈严格单调下降；GPU 距离计算和 1000x1000000 距离传输仍是主要固定成本，CPU partial_sort 的额外开销没有压倒该成本。所有 K 配置的 CPU/GPU exact recall 均为 1.0。

IVF 展示了清晰的速度-质量权衡：nprobe=1 达到 3509.281 QPS，但 recall@10 仅 0.0596；nprobe=16 为 595.972 QPS、recall@10=0.4729；nprobe=64 达到 0.9378 recall@10，但吞吐降至 153.481 QPS，低于精确检索 261.783 QPS。若重视质量，本实现的 nprobe=64 已没有速度优势；这说明 coarse quality、索引布局和 GPU Top-K 仍需优化。

索引构建耗时 4391.59 ms。工作区指标为程序显式 buffer 估算：精确 batch=128 为 976.625 MiB，IVF 随 nprobe 从约 488.390 MiB 增至 493.920 MiB；它不包含 CUDA context 与驱动开销。

10. 优化历程与已知瓶颈

初版选择统一的“距离越小越好”内部表示，使 L2 和内积复用相同 Top-K 逻辑；同时采用 batch distance matrix，避免逐 query 启动 kernel。第一版 IVF 会在 CPU 按 ID 打包候选向量并逐 query 上传，端到端 nprobe=16 仅约 11 QPS。定位后改为数据库一次上传并常驻 GPU、每次只传候选 ID，最终 nprobe=16 达到 595.972 QPS，召回率保持 0.4729。这一修改约提升 52 倍，并保留版本化索引加载。

当前主要瓶颈包括：每个线程独立扫描维度、精确检索 distance matrix 的 D2H 传输、CPU partial_sort、IVF 的 CPU coarse search，以及候选 ID 的随机访存。服务器没有安装 ncu/nsys，因此报告不虚构 profiler 结论。

11. 后续工作

- 使用 CUB block radix sort 或分层 heap 实现 GPU Top-K 与跨 block merge。
- 将数据库转为 tiled/转置布局，改善 warp 访存合并。
- 用 CUDA kernel 完成 IVF assignment 和 coarse search。
- 数据库与索引常驻 GPU，并用 pinned memory、stream 和 double buffering 重叠传输。
- 支持 FP16、cosine、IVF-PQ 和多 GPU shard。
- 使用 ncu 检查 memory throughput、occupancy、warp stall；使用 nsys 分析 H2D/kernel/D2H/CPU merge 时间线。